

5. 存储、权限与多租户

OpenViking 在存储层把内容、索引、身份边界和任务状态拆开管理。内容侧使用 VikingFS/AGFS 保存可读文件树；索引侧使用向量库保存摘要、分数和 metadata；服务侧通过 RequestContext 将账号、用户、Agent 和角色传递到每次读写与检索操作中。

空间	URI 示例	可见性与用途
资源空间	viking://resources/...	账号内共享知识资料，适合产品手册、制度、方案、资质库
用户记忆	viking://user/memories/...	按用户隔离的画像、偏好、实体和事件
Agent 记忆	viking://agent/memories/...	按 Agent 或 user+agent 范围隔离的案例、模式、工具和技能经验
技能空间	viking://agent/skills/...	Agent 可引用或执行的能力描述
会话空间	viking://session/{user}/{session_id}/...	单会话消息、归档、摘要和任务状态

- HTTP Client 请求头包含 X-API-Key、X-OpenViking-Account、X-OpenViking-User、X-OpenViking-Agent，用于服务端身份解析。
- resources 默认 owner_space 为空，表示账号共享；user、agent、session URI 会根据 UserIdentifier 派生 owner_space。
- 正式多租户模式可通过 root_api_key 和身份头启用；默认模式也保留 default account/user/agent 以便本地和嵌入式部署。
- 向量索引写入 account_id、owner_space、context_type、category、level 等字段，检索时以身份和空间过滤，避免跨用户或跨 Agent 泄漏。

6. API 与集成方式

集成形态	适用情况	说明
Embedded Python	AI 应用与 OpenViking 同进程部署	适合本地工具、实验环境、低延迟私有服务
HTTP Server	多个应用共享知识库服务	通过 FastAPI 路由提供资源、搜索、会话、文件系统等接口
Console / Admin UI	知识管理和运维操作	用于上传资料、查看任务、配置和调试
CLI / SDK	脚本化导入、测试、批处理	适合资料迁移、构建索引、自动化验证
Bot / MCP	连接具体 AI 终端或 Agent 框架	让客服机器人、投标助手等上层应用复用同一知识底座

能力	HTTP 路由/方法	主要用途
资源入库	POST /api/v1/resources	上传或引用资源，触发解析、摘要和索引
临时上传	POST /api/v1/resources/temp_upload	上传本地文件，返回 temp_file_id
语义检索	POST /api/v1/search/find	无会话语义搜索
会话检索	POST /api/v1/search/search	结合 session 的多意图检索